

Big Data Lab

Lab 4: Apache Spark 3.5+ Foundations & DataFrame API

Dr. Innocent Nyalala

IIT Madras Zanzibar
Department of Data Science and AI

Monday, 30 March 2026

DO.jpg

Today's Objectives

Theory (2:00–3:00)

- Spark architecture: driver, executors, cluster manager
- DAG execution model
- RDDs vs DataFrames vs Datasets
- Narrow vs wide transformations
- The Catalyst Optimizer
- Tungsten execution engine

Hands-On (3:10–4:30)

- Reading multiple formats (CSV, JSON, Parquet, Delta)
- Explicit schema vs inferSchema
- Complex transforms (explode, pivot, struct)
- Python UDFs and Pandas UDFs
- Broadcast joins vs sort-merge
- Caching and persistence
- Reading query execution plans

Project Proposals Due Today

DO.jpg

Apache Spark: The Post-Hadoop Era

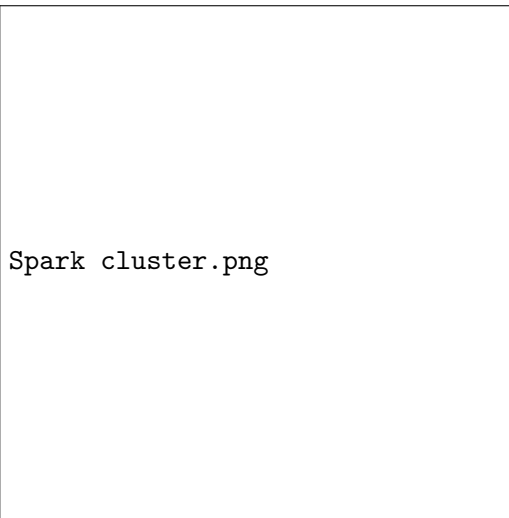
What is Apache Spark?

- A unified, lightning-fast analytics engine for large-scale data processing.
- Provides high-level APIs in Java, Scala, Python, and R.
- Built on an optimized engine that supports general execution graphs.

A Brief History

- **2009:** Created by Matei Zaharia at UC Berkeley's AMPLab.
- **2010:** Open-sourced under a BSD license.
- **2014:** Graduated as a Top-Level Apache Software Foundation Project.

Spark Cluster (High-Level)



The "Why": Solving the MapReduce Bottleneck

The Problem: Hadoop MapReduce

- Severely disk-bound: Wrote intermediate data back to physical disk after *every* single computation step.

The Spark Solution: In-Memory Computing

- Introduced **RDDs** (Resilient Distributed Datasets).
- Allows data to be cached in RAM across the cluster.
- Up to **100x faster** than MapReduce in memory, and 10x faster on disk.

The Unified Ecosystem

- **Spark SQL:** Batch Processing and heavy-duty ETL.
- **Structured Streaming:** Real-time data pipelines.
- **MLlib & GraphX:** Machine learning and graph computation.

DD.jpg

Spark Ecosystem (Overview)

sessions//lab04//slides/image1.png

DO.jpg

The Foundation: Resilient Distributed Datasets (RDDs)

What exactly is an RDD?

- **Resilient:** Fault-tolerant. If a node crashes, Spark rebuilds the lost data using the original lineage graph (the recipe of transformations).
- **Distributed:** Data is partitioned and spread across the cluster.
- **Dataset:** An immutable, partitioned collection of raw Java/Python objects.

The Catch with RDDs:

- They are **schema-less**. Spark sees them as a black box of objects.
- Because Spark doesn't understand the data inside, it **cannot optimize** your queries automatically.
- Heavy serialization overhead, especially if you are using PySpark.

Partition in RDD

`sessions/lab04/slides/rgd.jpg`

DO.jpg

The Modern Standard: Spark DataFrames

What is a DataFrame?

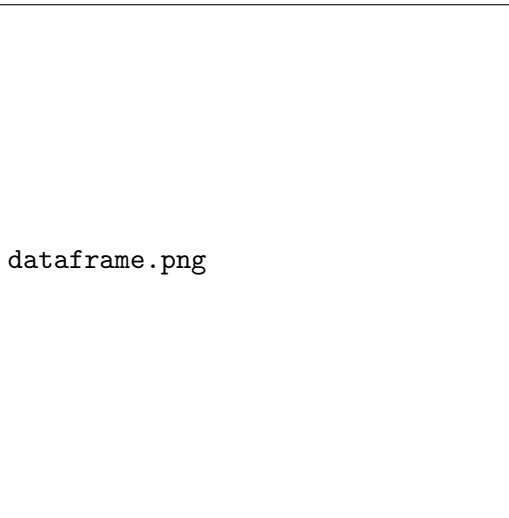
- Data organized into named columns, just like a table in a relational database.
- Built on top of RDDs, but with a **strict schema**.

Why did we switch to DataFrames?

- **Catalyst Optimizer:** Because Spark knows your schema, it rewrites your SQL/API calls into the most efficient physical execution plan possible.
- **Tungsten Engine:** Manages memory directly at the hardware level, bypassing the slow Java Garbage Collector.
- **Language Parity:** Python and Scala perform at the exact same speed because they both compile down to the same underlying JVM bytecode.

DO.jpg

DataFrames (Visual)



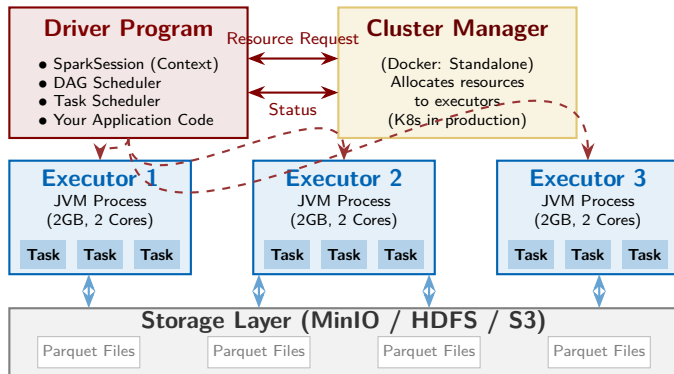
Core Differences: RDD vs. DataFrame

Feature	RDD	DataFrame
Data Format	Unstructured Java/Python objects	Structured, named columns (Relational)
Schema	Schema-less (Black box)	Schema-aware
Optimization	None (Developer's burden)	Automatic (Catalyst Optimizer)
Performance	Slower in Python (Serialization)	Extremely fast across all languages
Best Used For	Unstructured data (text, media), low-level control	Structured data (CSV, JSON, Parquet), SQL queries

Golden Rule: Always use DataFrames unless you have a highly specific, low-level operational need to use an RDD.

DD.jpg

Spark Architecture: Driver, Executors, Cluster Manager

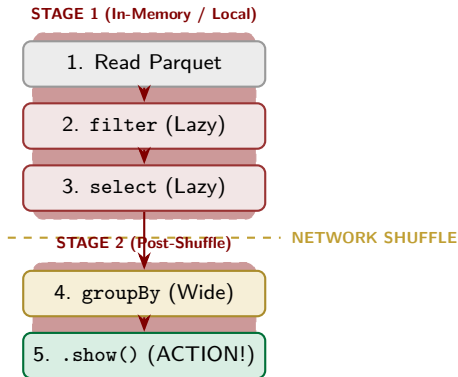


DO.jpg

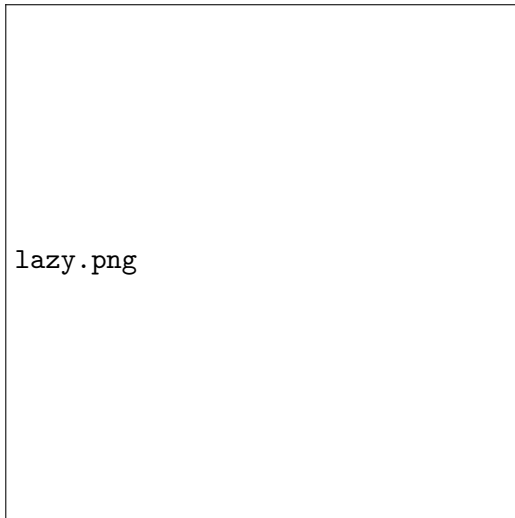
How Spark Executes: The DAG & Lazy Evaluation

Directed Acyclic Graph (DAG):

- **Transformations are Lazy:** filter, select, and groupBy do *nothing* immediately. Spark just adds a node to the DAG (taking notes).
- **Actions Trigger Execution:** count(), show(), or write force Spark to look at the whole DAG and execute it.
- **The Catalyst Optimizer:** Because it waits, Spark can rearrange and optimize your messy code *before* running it.
- **Shuffles = Stage Boundaries:** Any operation that forces data to move across the network (like a groupBy) cuts the DAG into separate **Stages**.



Lazy Evaluation (Visual)



The Blueprint: Directed Acyclic Graph (DAG)

What is a DAG?

Think of the DAG as Spark's **master blueprint** for your data pipeline. It is a mathematical mapping of your logical execution plan.

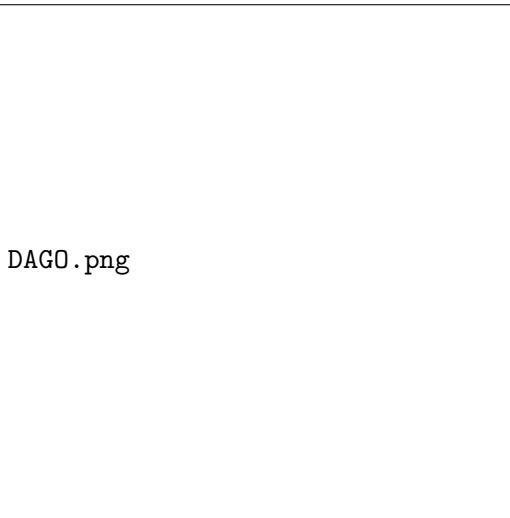
- **Directed:** Data flows in exactly one direction—from the source (e.g., a Parquet file) to the final output. It never goes backward.
- **Acyclic:** There are no loops. You cannot get stuck in an infinite circle of operations.
- **Graph:** A structure made of **nodes** (transformations like `filter`) connected by **edges** (the flow of data).

The Post-Hadoop Upgrade

In MapReduce, every job was isolated and had to read/write to physical disk. Spark builds a single, unified DAG of your *entire* workflow in memory before taking a single step.

DO.jpg

DAG Example



The Superpower: Lazy Evaluation

Spark is a Master Procrastinator.

It waits until the absolute last second to do any actual work.

- **Transformations (Lazy):** `select()`, `filter()`, `join()`. Spark does *zero computation* here. It just adds a node to the DAG (taking notes).
- **Actions (Eager):** `count()`, `write()`, `show()`. This tells Spark: "I need the result *right now*." Execution begins.

Why is this brilliant?

- **The Catalyst Optimizer:** Because Spark waits, it can look at your entire DAG and rewrite your inefficient code *before* running it.
- **Example:** You join a 1-billion row table, then filter for `country = 'Tanzania'`.
 - *Eager execution:* Joins 1B rows first (crashing the cluster).
 - *Lazy evaluation:* Rewrites the plan to filter *before* the join. Turns a 5-hour job into a 5-second job.

DO.jpg

Spark Execution at a Glance

SparrksS.png

DO.jpg

RDDs vs DataFrames vs Datasets

Feature	RDD	DataFrame	Dataset (Scala)
Level	Lowest-level API	High-level, tabular	Typed high-level
Type safety	Compile-time	Runtime schema	Compile-time
Catalyst optimizer	No	Yes	Yes
Tungsten memory	No	Yes	Yes
Python support	Yes	Yes	JVM only
Ease of use	Complex	Easy	Medium
Performance	Slowest	Fast	Fast

Bottom Line for This Course (PySpark)

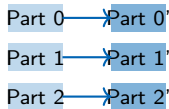
Always use DataFrames. Datasets require Scala/Java. RDDs are legacy — only use them when you need custom partitioning or operations that DataFrames cannot express (extremely rare). The Catalyst optimizer makes DataFrames 2–10× faster than equivalent RDDs.

DO.jpg

Narrow vs Wide Transformations

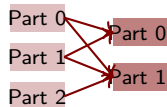
Narrow (no shuffle):

- Each output partition depends on **ONE** input partition.
- No network data transfer between executors.
- Multiple narrow transforms are **pipelined** into one Stage.
- Examples: filter, select, withColumn, map, flatMap, union.



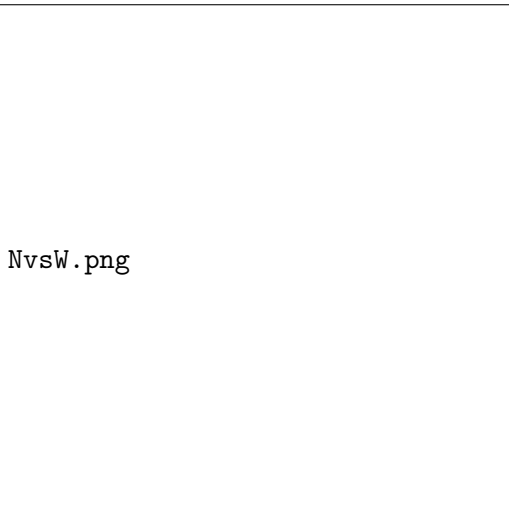
Wide (shuffle required):

- Output partitions may need data from **MANY** input partitions.
- Requires **network shuffle** (expensive!).
- Always creates a new **Stage boundary** in the DAG.
- Examples: groupBy, join, orderBy, distinct, repartition.



Rule: Minimise wide transformations. Two consecutive groupBy operations = two full network shuffles. Combine into one when possible to cut 50% of shuffle cost.

Narrow vs. Wide Transformations



Stage Boundaries: What Triggers a Shuffle?

A new Stage starts when Spark sees:

- `groupBy` / `agg` — hash shuffle
- `join` — sort-merge or broadcast
- `orderBy` / `sort` — total sort shuffle
- `distinct` — hash shuffle
- `repartition(N)` — round-robin shuffle
- `pivot` — multiple aggregations + shuffle

Shuffle cost = $\# \text{partitions} \times \text{data per partition} \times \text{network latency}$.

Default `spark.sql.shuffle.partitions` = 200.

For our lab: set to **4–20** for datasets < 5 GB.

How to spot shuffles in the Spark UI:

- **Stages tab:** “Shuffle Read/Write” cols
- **SQL tab:** look for Exchange nodes
- A stage writes data → next stage reads it

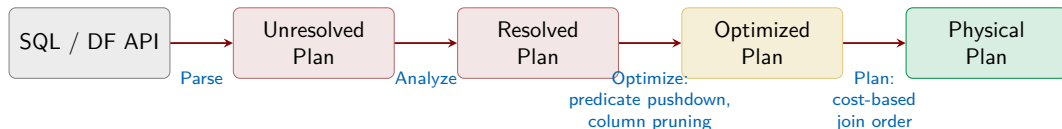
Quick Tuning

Before any query in today's lab:

```
spark.conf.set(
  "spark.sql.shuffle.partitions", "20")
```

This alone speeds up queries 3–5× on small datasets.

The Catalyst Optimizer: Why Spark SQL Is Fast



Predicate Pushdown

`WHERE year=2024` applied *before* reading data. Only relevant Parquet row groups are read from disk.

Column Pruning

`SELECT a, b` reads only those columns from Parquet. Other columns never loaded into memory.

Join Reordering

Catalyst reorders joins to minimise data shuffled. Uses table statistics to choose the optimal physical plan.

Project Tungsten Memory

The JVM Nightmare

- **Object Overhead:** A simple 4-byte string in Java can consume 48 bytes of RAM due to object headers and padding.
- **Garbage Collection (GC):** When memory gets full, the JVM pauses *everything* to clean up. In distributed processing, these pauses can freeze a cluster for minutes.

The Tungsten Solution

- **Off-Heap Memory:** Tungsten bypasses the JVM. It manages memory directly, much like C or C++. **No GC pauses!**
- **Binary Processing:** Stores data in a highly compact, raw binary format.
- **Direct Execution:** Spark can filter and sort this binary data directly *without* deserializing it back into Java objects.

The Result

Spark operates at “bare metal” speeds. This is why PySpark users get the exact same performance as Scala users- the Python code is just an API call that triggers the Tungsten engine under the hood.

DO.jpg

The Tungsten Execution Engine

Why Tungsten? JVM garbage collection was the bottleneck:

- Java objects: 16–32 bytes overhead per object header.
- GC pauses could last seconds, stalling all tasks.
- Tungsten moves data **off the JVM heap**.

Three Tungsten innovations:

1. **Off-heap memory management:**
Direct memory via `sun.misc.Unsafe`. Zero GC overhead.
2. **Whole-Stage Code Generation:**
Catalyst generates Java bytecode at compile time — runs a tight loop instead of interpreting operators row-by-row.
3. **Vectorised columnar I/O:**
Reads Parquet/ORC in batches of 10,000 rows (not 1 row at a time). CPU cache-aware data

Practical Impact

Tungsten is **completely transparent** to PySpark code. Under the hood:

- 2–10× faster than Spark 1.x (pre-Tungsten)
- GC time drops from ~40% to <5% of task time
- Vectorised Parquet reads: 100M rows/s per core

Check in Spark UI

In the **Stages** tab: look at the **GC Time** column.
Healthy: <5% of Task Time.

Unhealthy: >20% — memory pressure; consider increasing executor memory or using `DISK_ONLY` persistence.

DO.jpg

Lazy Evaluation: See It In Action

```
import time
from pyspark.sql.functions import col, count, avg, round as spark_round

# STEP 1 -- All of these are LAZY (zero computation, check Spark UI: 0 jobs)
df_raw = spark.read.option("header", "true").option("inferSchema", "true")
\
    .csv("s3a://warehouse/raw/orders.csv")
df_filtered = df_raw.filter(col("status") == "completed")
df_selected = df_filtered.select("order_id", "country", "category", "amount")
df_grouped = df_selected.groupBy("country").agg(
    count("order_id").alias("n_orders"),
    spark_round(avg("amount"), 2).alias("avg_value")
)
print("Transformations defined (lazy). Spark UI shows 0 jobs.")

# STEP 2 -- First ACTION triggers the entire optimized plan in one go
t0 = time.time()
df_grouped.show(10)      # <-- this is the ACTION
print(f"Completed in {time.time()-t0:.2f}s. Check Spark UI: 1 job, 2 stages.")

# STEP 3 -- Read the physical plan WITHOUT running again
df_grouped.explain(mode="simple")
```

DO.jpg

Reading Multiple Data Formats

```
# 1. CSV -- human-readable but slowest (no embedded schema, no column stats)
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss") \
    .csv("s3a://warehouse/raw/orders.csv")

# 2. JSON -- good for nested / semi-structured data
df_json = spark.read.json("s3a://warehouse/raw/products.json")

# 3. Parquet -- columnar, compressed, fast; preferred for analytics
df_parquet = spark.read.parquet("s3a://warehouse/silver/orders/")

# 4. Delta Lake -- ACID + Parquet + transaction log
df_delta = spark.read.format("delta").load("s3a://warehouse/gold/sales/")

# 5. Glob: read multiple monthly files at once
df_many = spark.read.parquet("s3a://warehouse/silver/orders/year=2025/*/")

# Compare
for name, df in [("CSV", df_csv), ("JSON", df_json),
                 ("Parquet", df_parquet), ("Delta", df_delta)]:
```

Explicit Schema vs inferSchema: Why It Matters

```
from pyspark.sql.types import (StructType, StructField, StringType,
                                DoubleType, IntegerType, TimestampType)

import time

# BAD: inferSchema reads the ENTIRE file twice (once to infer, once to load)
t0 = time.time()
df_infer = spark.read.option("header","true").option("inferSchema","true") \
    .csv("s3a://warehouse/raw/orders.csv")
df_infer.count()
print(f"inferSchema: {time.time()-t0:.2f}s    (2-pass read)")

# GOOD: Define schema explicitly -- one-pass read, correct types guaranteed
order_schema = StructType([
    StructField("order_id",      StringType(),      nullable=False),
    StructField("customer_id",   StringType(),      nullable=True),
    StructField("country",       StringType(),      nullable=True),
    StructField("category",      StringType(),      nullable=True),
    StructField("quantity",      IntegerType(),     nullable=True),
    StructField("unit_price",     DoubleType(),     nullable=True),
    StructField("order_date",     TimestampType(),  nullable=True),
    StructField("status",        StringType(),      nullable=True),
])
```

DO.jpg

Complex Transforms: Explode, Struct, Higher-Order

```
from pyspark.sql.functions import explode, struct, transform, split

# EXPLODE: one row with array column -> one row per array element
orders_with_tags = spark.createDataFrame([
    (1, "Alice", "Electronics,Home,Gadgets"),
    (2, "Bob", "Furniture,Office"),
    (3, "Carol", "Clothing"),
], ["order_id", "customer", "tags_str"])

orders_with_tags \
    .withColumn("tag", explode(split(col("tags_str"), ","))) \
    .select("order_id", "customer", "tag") \
    .show()

# STRUCT: group multiple columns into a nested struct
df = spark.createDataFrame([(1, -6.792, 39.208)], ["id", "lat", "lon"])
df.withColumn("gps", struct(col("lat"), col("lon"))) \
    .select("id", "gps", col("gps.lat").alias("latitude")).show()

# HIGHER-ORDER: apply a lambda to every element of an array (Spark 2.4+)
prices = spark.createDataFrame([(1, [100.0, 250.0, 75.0])], ["id", "prices"])
prices.withColumn("discounted",
```

Complex Transforms: Pivot and Unpivot

```
from pyspark.sql.functions import sum as spark_sum

sales = spark.createDataFrame([
    ("Jan", "Kenya", 4500.0), ("Jan", "Tanzania", 3200.0), ("Jan", "Uganda", 1800.0),
    ("Feb", "Kenya", 5100.0), ("Feb", "Tanzania", 3800.0), ("Feb", "Uganda", 2100.0),
    ("Mar", "Kenya", 4900.0), ("Mar", "Tanzania", 4100.0), ("Mar", "Uganda", 2400.0),
], ["month", "country", "revenue"])

# PIVOT: rows -> columns (one column per unique value of 'country')
pivot_df = sales.groupBy("month") \
    .pivot("country", ["Kenya", "Tanzania", "Uganda"]) \
    .agg(spark_sum("revenue"))
pivot_df.show()
# Output:  month | Kenya | Tanzania | Uganda
#           Jan   | 4500.0 | 3200.0   | 1800.0 ...

# UNPIVOT (stack): columns -> rows -- reversal of pivot (Spark 3.4+)
unpivot_df = pivot_df.unpivot(
    ids=["month"],
    values=["Kenya", "Tanzania", "Uganda"],
    variableColumnName="country",
    valueColumnName="revenue"
```

DO.jpg

Python UDFs: Powerful but Slow

```
from pyspark.sql.functions import udf, when
from pyspark.sql.types import StringType
import time

# Python UDF -- each row crosses Python-JVM boundary (serialisation overhead)
@udf(returnType=StringType())
def classify_order(amount: float) -> str:
    if amount is None:      return "Unknown"
    if amount >= 1000.0:    return "High-Value"
    if amount >= 200.0:     return "Standard"
    return "Low-Value"

df = spark.table("orders_typed")

t0 = time.time()
df.withColumn("tier", classify_order(col("order_total"))).count()
print(f"Python UDF:      {time.time()-t0:.2f}s")

# COMPARE: built-in Spark functions (pure JVM, no Python overhead)
t0 = time.time()
df.withColumn("tier",
    when(col("order_total") >= 1000.0, "High-Value")
```

DO.jpg

Pandas UDFs: Vectorised Python via Apache Arrow

```
import pandas as pd
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType, StringType
import time

# Pandas UDF -- processes entire Series (column batch) at once using Arrow
# No row-by-row Python-JVM overhead!

@pandas_udf(DoubleType())
def compute_margin(revenue: pd.Series, cost: pd.Series) -> pd.Series:
    return ((revenue - cost) / revenue * 100).round(2)

@pandas_udf(StringType())
def clean_country(s: pd.Series) -> pd.Series:
    return s.str.strip().str.title()

# Apply Pandas UDFs
df = spark.table("orders_typed")
t0 = time.time()
df.withColumn("country_clean", clean_country(col("country"))) \
    .groupBy("country_clean") \
    .agg(count("order_id").alias("n"),
```

DO.jpg

Join Strategies: Choosing the Right Algorithm

Strategy	When Used	Shuffle?	Best For
Broadcast Hash	Small table ≤ 10 MB	No shuffle	Fact $\times$ small dimension
Shuffle Hash	Medium tables	Both sides shuffle	Medium, no sort
Sort Merge Join	Large tables (default)	Both shuffle + sort	Large $\times$ Large
Cartesian	No join key	Enormous cost	Avoid in production

Broadcast Join Threshold

Config: `spark.sql.autoBroadcastJoinThreshold`
Default: **10 MB**. Set -1 to disable.
Force broadcast: `broadcast(df)` hint.
UI SQL tab: look for `BroadcastHashJoin`.

AQE Dynamic Join Switch

Spark 3+ AQE can **automatically convert** a planned `SortMergeJoin` to `BroadcastHashJoin` at runtime if one side turns out smaller than the threshold after filtering.

Enable: `spark.sql.adaptive.enabled=true`

do.jpg

Broadcast Joins in Practice

```
from pyspark.sql.functions import broadcast
import time

df_orders = spark.table("orders_typed")    # large table: 100K+ rows

# Small lookup table: country metadata (~50 rows)
country_meta = spark.createDataFrame([
    ("Kenya", "KE", "KES", "Africa/Nairobi"),
    ("Tanzania", "TZ", "TZS", "Africa/Dar_es_Salaam"),
    ("Uganda", "UG", "UGX", "Africa/Kampala"),
    ("Ghana", "GH", "GHS", "Africa/Accra"),
    ("Nigeria", "NG", "NGN", "Africa/Lagos"),
], ["country", "iso2", "currency", "timezone"])

# BAD: Sort-Merge Join -- shuffles BOTH tables across the network
t0 = time.time()
df_orders.join(country_meta, "country").count()
print(f"SortMerge Join: {time.time()-t0:.2f}s")

# GOOD: Broadcast Join -- sends small table to every executor once, no shuffle
t0 = time.time()
df_orders.join(broadcast(country_meta), "country").count()
```

DO.jpg

Caching and Persistence

```
from pyspark import StorageLevel
import time

df_base = spark.read.parquet("s3a://warehouse/silver/orders/") \
    .filter(col("status") == "completed") \
    .withColumn("order_total", col("quantity") * col("unit_price"))

# WITHOUT cache: Parquet is read TWICE from MinIO (2x network I/O)
t0 = time.time()
df_base.groupBy("country").count().collect()
df_base.groupBy("category").agg(avg("order_total")).collect()
print(f"No cache: {time.time()-t0:.2f}s   (2 full scans)")

# WITH cache: read Parquet ONCE, store in executor memory
df_cached = df_base.cache()
df_cached.count()      # materialise the cache (this triggers the first scan)

t0 = time.time()
df_cached.groupBy("country").count().collect()
df_cached.groupBy("category").agg(avg("order_total")).collect()
print(f"With cache: {time.time()-t0:.2f}s   (reads from RAM)")
```

DO.jpg

Writing to Multiple Output Formats

```
from pyspark.sql.functions import date_format, sum as spark_sum

result = spark.table("orders_typed") \
    .withColumn("month", date_format("order_date", "yyyy-MM")) \
    .groupBy("month", "country", "category") \
    .agg(
        count("order_id").alias("n_orders"),
        spark_round(spark_sum("order_total"), 2).alias("total_revenue"),
        spark_round(avg("order_total"), 2).alias("avg_order_value")
    )

# 1. Parquet: best for analytics (columnar, compressed, splittable)
result.write.mode("overwrite").partitionBy("month") \
    .parquet("s3a://warehouse/gold/summary_parquet/")

# 2. Delta Lake: ACID, schema evolution, time travel
result.write.format("delta").mode("overwrite").partitionBy("month") \
    .save("s3a://warehouse/gold/summary_delta/")

# 3. CSV: human-readable for sharing with non-Spark tools
result.coalesce(1).write.mode("overwrite").option("header", "true") \
    .csv("s3a://warehouse/gold/summary_csv/")
```

Reading Execution Plans: explain()

```
df = spark.read.parquet("s3a://warehouse/silver/orders/")

query = df.filter(col("country") == "Kenya") \
           .filter(col("status") == "completed") \
           .groupBy("category") \
           .agg(count("*").alias("n"), avg("order_total").alias("avg_val"))

# explain() modes:
query.explain()                # simple physical plan only
query.explain("extended")      # all 4 plan phases
query.explain(mode="formatted") # tree + per-node metrics (most useful)

# Annotated example output:
# == Physical Plan ==
# AdaptiveSparkPlan (AQE enabled)
# +- HashAggregate [category], [count(1), avg(order_total)] <- agg step 2
#   +- Exchange hashpartitioning(category, 20) <- SHUFFLE!
#     +- HashAggregate [category] <- agg step 1
#       +- Filter (country = Kenya AND status = completed) <- pushed down
#         +- FileScan parquet [country,status,category,order_total]
#           PushedFilters: [IsNull(country), ...]
#
```

DO.jpg

Reading the Spark UI: Your Debugging Dashboard

Key Tabs:

- **Jobs:** Overview. Click a job to see its stages.
- **Stages:** Shuffle Read/Write, GC Time, Task duration. High shuffle = opportunity to optimize.
- **SQL:** Visual plan DAG per query. Key nodes:
 - Exchange = shuffle (look for this!)
 - BroadcastHashJoin = fast join
 - Filter below FileScan = predicate pushdown ✓
 - Filter above FileScan = pushdown failed ×
- **Storage:** Cached DataFrames and memory usage.
- **Environment:** All active Spark configurations.

Metric		Action if High
Shuffle	Read	Tune shuffle partitions
>1 GB		
GC Time >20%		Increase executor memory
Task time spread		Data skew — salt the key
Spill to Disk		More memory or smaller partitions
Rows in	>> Rows	Predicate pushdown
out		working?

Lab URLs:

Spark Master: localhost:8080
Live job UI: localhost:4040
(only visible during active job)

00.jpg

Common Pitfalls and Best Practices

Anti-Patterns to Avoid

- **inferSchema on large files:** 2-pass read, slow. Define schema explicitly.
- **collect() on large DataFrames:** Pulls all data to driver RAM. Use `show()` or `write`.
- **Python UDFs:** Slow! Always search `pyspark.sql.functions` first.
- **Forgetting to unpersist:** Cached DFs fill memory silently. Call `.unpersist()`.
- **shuffle.partitions=200:** Too high for small data. Set to 4–20 for our lab cluster.
- **repartition() to shrink count:** Use `coalesce()` — it avoids a full shuffle.

Best Practices

- Define schemas explicitly in production.
- Cache DataFrames used ≥ 2 times.
- Use `broadcast()` for small lookup tables.
- Set `spark.sql.shuffle.partitions` to match data size.
- Enable AQE (`spark.sql.adaptive.enabled=true`).
- Prefer Parquet or Delta over CSV.
- Use `explain()` before running expensive queries.
- Read the Spark UI after every slow job.
- Partition output data by date for time-series queries.

DO.jpg

Assignment 4: Spark Foundations (100 marks)

1. **(15)** Setup: Verify the Docker stack. Screenshot Spark Master UI showing both workers. Create a SparkSession with ≥ 3 custom configs. Print the Spark version.
2. **(30)** Dataset analysis: Choose a real dataset ($\geq 100K$ rows). Apply ≥ 8 operations: filter, join, groupBy, window function, explode/pivot, explicit schema, Python UDF, Pandas UDF. Write results to Delta Lake on MinIO.
3. **(20)** Join strategies: Join two tables using (a) SortMerge and (b) broadcast. Show `explain()` for both. Report execution time difference. Screenshot both SQL plans in Spark UI.
4. **(15)** Caching benchmark: Run the same groupBy query (a) without cache and (b) with `.cache()`. Measure time with `time.time()`. Report speedup.
5. **(20)** Plan analysis: Write 3 queries. For each, call `explain(mode="formatted")`. Identify shuffles, predicate pushdown success/failure, column pruning.

DO.jpg

Next Week — Lab 5: Spark SQL + Performance Tuning

What we will cover in Lab 5:

- Advanced Spark SQL: CTEs, lateral views, PIVOT/UNPIVOT
- Window functions: NTILE, FIRST/LAST_VALUE, QUALIFY
- Bucketing vs partitioning
- AQE deep dive: 3 automatic optimisations
- Data skew: detection + the salting technique
- Benchmarking: before vs after optimisation

Come Prepared!

Bring a **slow query** from Assignment 4 that took >30 seconds. We will diagnose it live in the Spark UI and apply optimizations to speed it up **10–100×**.

Reading: Karau & Warren, *High Performance Spark*, Ch. 3–4.

DO.jpg